

SQL Database

 systemdesignschool.io/fundamentals/sql-database



We explored three storage categories: SQL, NoSQL, and Blob storage. SQL databases enforce structure and relationships. This article explains why that structure matters and when to use it.

The Problem Without Structure

A bank processes account transfers. Customer A transfers \$100 to Customer B. The operation requires two steps: deduct \$100 from A, add \$100 to B. A network error occurs after the deduction but before the addition. Account A loses \$100. Account B gains nothing. The money vanishes.

File-based storage can't prevent this. Neither can most key-value stores. You need **coordinated operations that either fully succeed or fully fail**. This guarantee is the foundation of SQL databases.

Partial Failure Without Transactions

Step 1: Initial State



Step 2: Deduct \$100 from A ✓



Network Error!

Step 3: Add \$100 to B ✗ (Failed)



Inconsistent State!

- Total before: \$700
- Total after: \$600
- Missing: \$100

Without transactions, partial failures corrupt data permanently



Why SQL Exists

Production systems face three related problems that unstructured storage cannot solve.

First, race conditions corrupt data. Two ticket buyers read `seats_available=1` simultaneously. Both proceed to purchase. The second write overwrites the first. Two customers hold tickets for the same seat. File systems and simple databases let both writes succeed.

Second, partial failures create inconsistent state. An e-commerce order involves inventory decrement, payment processing, and order creation. If payment succeeds but inventory update fails, the store oversells. Manual reconciliation takes hours or days.

Third, unstructured data drifts over time. One engineer stores phone numbers as integers. Another uses strings with dashes. A third includes country codes. Queries break. Reports fail. Schema enforcement prevents this drift.

SQL databases solve these problems through structured schemas and transactional guarantees.

How SQL Databases Work

SQL organizes data into tables. Each table represents an entity type: customers, orders, products. Rows represent individual instances. Columns define attributes with specific data types.

```
CREATE TABLE Customers ( CustomerID int PRIMARY KEY, Name varchar(255) NOT NULL, Email varchar(255) UNIQUE, Balance decimal(10,2) DEFAULT 0.00 );
```

This schema enforces rules before data enters the database. `CustomerID` must be an integer and unique. `Name` cannot be null. `Email` must be unique across all rows. `Balance` must be a decimal with exactly two places. Attempting to insert `"invalid"` into `CustomerID` fails immediately.

Primary Keys. Every table needs a unique identifier for each row. `CustomerID` serves this role. The database creates an index on this column, enabling fast lookups. Without a primary key, you cannot reliably update or delete specific rows.

Tables and Relationships

Tables connect through foreign keys. An `Orders` table references `Customers` through `CustomerID`:

```
CREATE TABLE Orders ( OrderID int PRIMARY KEY, CustomerID int NOT NULL, Amount decimal(10,2), OrderDate timestamp, FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) );
```

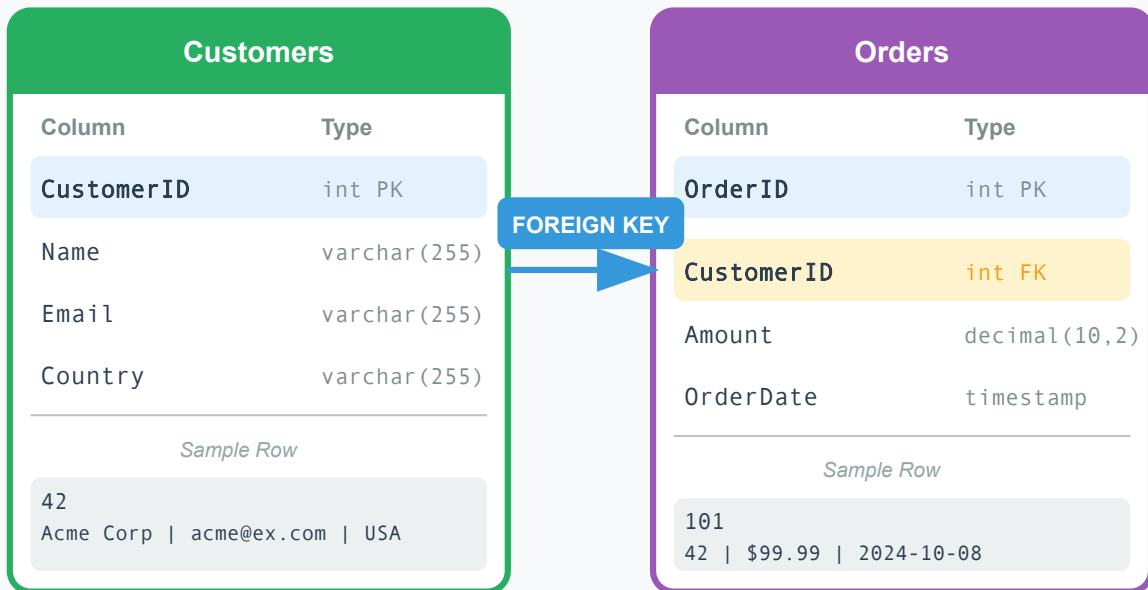
The foreign key constraint enforces referential integrity. You cannot create an order for a non-existent customer. Attempting `INSERT INTO Orders (CustomerID) VALUES (999)` fails if customer 999 doesn't exist. This prevents orphaned records.

Relationships enable complex queries across multiple tables. Find all orders for a specific customer:

```
SELECT o.OrderID, o.Amount, o.OrderDate FROM Orders o JOIN Customers c ON o.CustomerID = c.CustomerID WHERE c.Email = 'user@example.com';
```

The database uses indexes to execute this join efficiently, avoiding full table scans.

Tables and Foreign Key Relationships



Orders.CustomerID must reference an existing Customers.CustomerID



ACID Transactions

ACID properties guarantee reliable transaction processing. **Atomicity** treats multiple operations as a single unit. **Consistency** enforces schema rules and constraints. **Isolation** prevents concurrent transactions from interfering. **Durability** ensures committed data survives crashes.

Consider the bank transfer example. Wrap both operations in a transaction:

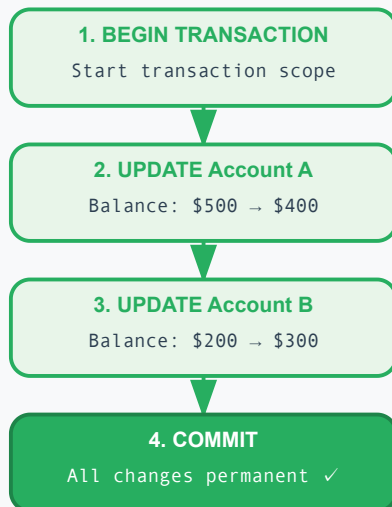
```
BEGIN TRANSACTION; UPDATE Accounts SET Balance = Balance - 100 WHERE AccountID = 'A' AND Balance >= 100; UPDATE Accounts SET Balance = Balance + 100 WHERE AccountID = 'B'; COMMIT;
```

Atomicity ensures both updates execute or neither does. If the second update fails, the database rolls back the first. The network error scenario becomes impossible. **Consistency** enforces the balance check; accounts can't go negative. **Isolation** prevents other transactions from reading intermediate state. Different [isolation levels](#) trade off correctness for performance. **Durability** writes committed changes to disk before acknowledging success.

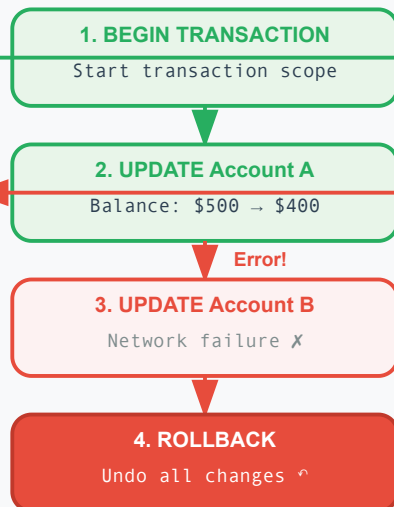
Write-Ahead Log (WAL). Databases write changes to a sequential log before modifying data files. Each log entry contains the old value and new value. On crash, the database replays the log to restore consistent state. This enables durability without sacrificing write performance.

ACID Transaction: Atomicity Guarantees

Success Path: All Operations Complete



Failure Path: Automatic Rollback



Database State

Initial
A: \$500 | B: \$200
Total: \$700

After Success ✓
A: \$400 | B: \$300
Total: \$700 ✓

After Rollback ↶
A: \$500 | B: \$200
Total: \$700 ✓

ACID Properties in Action

- Atomicity:** Both updates succeed or both roll back
- Consistency:** Balance checks enforced (no negative values)
- Isolation:** Other transactions don't see intermediate state



Basic SQL Operations

SQL provides four fundamental operations. **SELECT** retrieves data. **INSERT** adds rows. **UPDATE** modifies existing data. **DELETE** removes rows.

Retrieve customers in the USA:

```
SELECT Name, Email FROM Customers WHERE Country = 'USA';
```

Add a new customer:

```
INSERT INTO Customers (Name, Email, Country) VALUES ('Acme Corp',  
'contact@acme.com', 'USA');
```

Update contact information:

```
UPDATE Customers SET Email = 'new@acme.com' WHERE CustomerID = 42;
```

Delete a customer:

```
DELETE FROM Customers WHERE CustomerID = 42;
```

These operations combine with transactions to enable complex workflows while maintaining consistency.

When to Use SQL Databases

SQL databases excel when data relationships and integrity matter more than flexibility.

Financial systems require atomic multi-account updates. A payment processing system debits one account and credits another. These operations must succeed together or fail together. SQL transactions provide this guarantee. [Stripe's payment infrastructure](#) uses PostgreSQL to ensure payment consistency.

E-commerce platforms manage inventory, orders, and customer data with strict referential integrity. An order must reference an existing customer and product. Inventory decrements must match order creation. Foreign key constraints enforce these relationships. Attempting to delete a product with pending orders fails automatically.

Healthcare systems store patient records, appointments, and prescriptions. Privacy regulations demand audit trails showing who accessed which records when. SQL databases provide row-level permissions and transaction logs. Regulatory compliance often requires this level of control.

Reporting systems execute complex analytical queries. Calculate revenue by product category and region for the last quarter:

```
SELECT p.Category, c.Region, SUM(o.Amount) as Revenue FROM Orders o JOIN
Customers c ON o.CustomerID = c.CustomerID JOIN Products p ON o.ProductID =
p.ProductID WHERE o.OrderDate >= '2024-10-01' GROUP BY p.Category, c.Region;
```

SQL's query optimizer handles joins and aggregations efficiently. Building this query in a key-value store requires application-level processing.

Popular SQL Databases

PostgreSQL offers advanced features including custom data types, full-text search, and JSON support. It handles both transactional workloads and complex analytics. [Instacart uses PostgreSQL](#) for order processing at scale.

MySQL prioritizes simplicity and read performance. Many web applications use MySQL for user data and content management. Its replication features support read-heavy workloads.

Oracle Database provides enterprise features like advanced partitioning and multi-version concurrency control. Large corporations use Oracle for mission-critical applications requiring high availability.

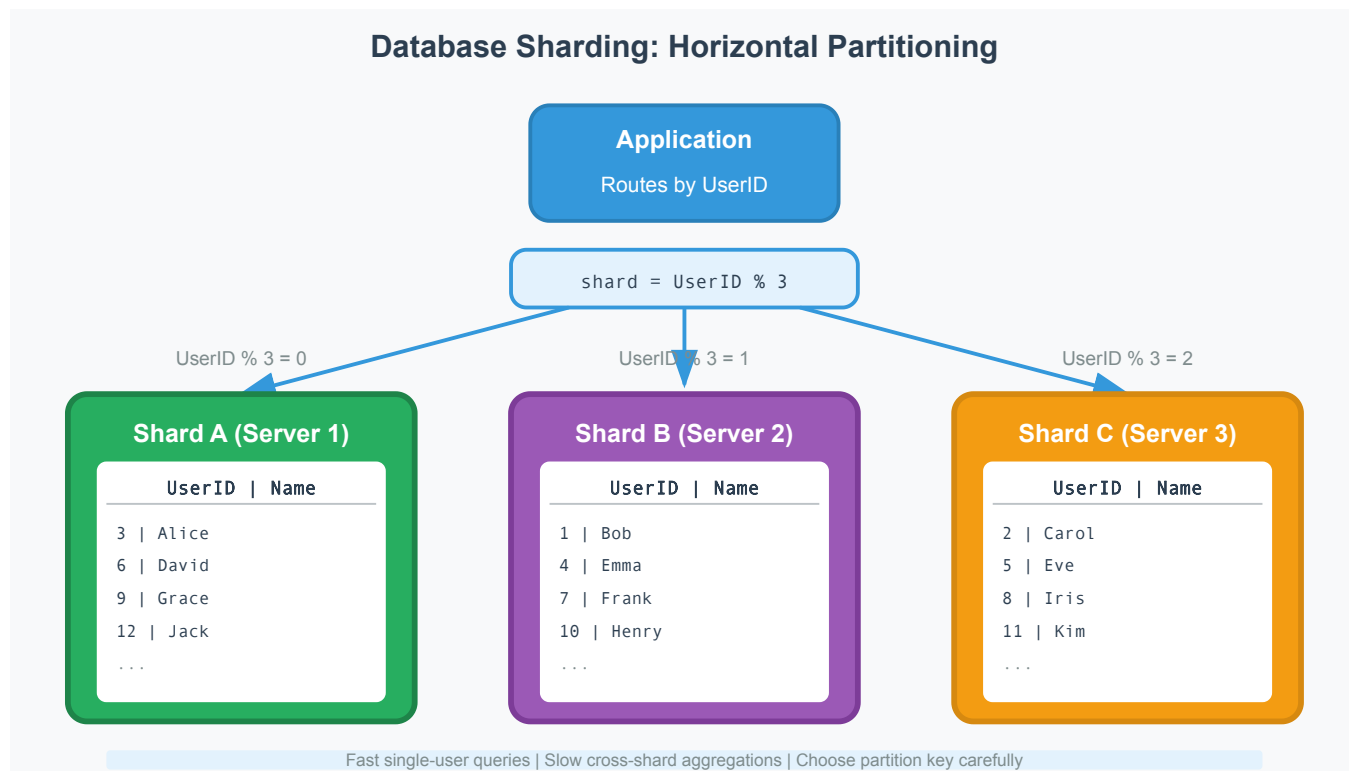
Challenges with SQL Databases

Scaling SQL databases requires careful planning. A single server handles limited throughput. Adding capacity means choosing between vertical and horizontal scaling.

Vertical scaling adds CPU, RAM, and faster disks to existing servers. This approach is simple but expensive. Hardware costs grow exponentially. A server with 512GB RAM costs more than twice a 256GB server. Eventually you hit physical limits.

Horizontal scaling distributes data across multiple servers through [sharding](#). This requires partitioning data by some key. User data might partition by user ID ranges. Orders might partition by creation date. Queries spanning partitions become expensive. Joins across servers require network round trips and application-level coordination.

Database Sharding. Partition data across multiple independent databases. Each shard handles a subset of data. User IDs 1-1000000 go to shard A, 1000001-2000000 to shard B. Queries for a single user remain fast. Cross-shard aggregations require querying all shards and merging results. Choose partition keys carefully to minimize cross-shard operations. We cover sharding strategies in detail in the database partitioning section.



[Replication](#) helps with read scaling. Write to a primary server. Replicas copy changes and serve reads. This works when reads vastly outnumber writes. Write-heavy workloads still bottleneck on the primary.

Schema changes become complex at scale. Adding a column to a billion-row table locks the table for hours. Production systems use online migration tools that copy data incrementally while serving traffic. [GitHub's gh-ost](#) demonstrates this approach.

Microservices architectures complicate SQL usage. Traditional monoliths use a single shared database. Microservices favor a database per service. This improves independence but loses transactional guarantees across services. Implementing distributed transactions or sagas adds complexity.